Bước 36 - GraphQL Aliases, cụ thể như Bước 35

Câu hỏi chính:

> Alias thuộc GraphQL hay Rails? Có nằm trong request của client không?

Câu trả lời:

Alias là cú pháp của GraphQL và được viết trực tiếp trong GraphQL document mà client gửi lên /graphql. Rails không cần tạo class hay resolver riêng cho alias.

1. Alias là gì?

Alias cho phép client:

```
1. đ□i tên key trong response
2. gọi cùng một field nhi□u l□n với arguments khác nhau
```

Cú pháp:

```
aliasName: fieldName
```

Ví dụ:

```
query GetProject($id: ID!) {
  project(id: $id) {
    projectName: name
  }
}
```

Schema Rails vẫn có:

```
field :name,
      String,
      null: false
```

Client nhận:

```
{
  "data": {
    "project": {
      "projectName": "CRM"
    }
  }
}
```

Không phải:

```
{
  "name": "CRM"
}
```

2. Alias thuộc GraphQL, không thuộc Rails

Client viết:

```
projectName: name
```

Rails không cần:

```
field :project_name
```

Không cần:

```
def project_name
end
```

Schema chỉ cần field gốc:

```
field :name,
      String,
      null: false
```

GraphQL engine hiểu:

```
projectName
→ response key

name
→ field thật trong schema
```

3. Trường hợp quan trọng nhất: cùng một field, arguments khác nhau

Giả sử bạn muốn lấy:

```
TODO tasks
IN_PROGRESS tasks
DONE tasks
```

Nếu viết:

```
query GetTaskGroups($projectId: ID!) {
  tasks(
    projectId: $projectId
    filter: {
      status: [TODO]
    }
  ) {
    nodes {
      id
    }
  }

  tasks(
    projectId: $projectId
    filter: {
      status: [DONE]
    }
  ) {
    nodes {
      id
    }
  }
}
```

GraphQL sẽ conflict vì cả hai cùng có response key:

```
tasks
```

nhưng arguments khác nhau.

Alias giải quyết:

```
query GetTaskGroups($projectId: ID!) {
  todoTasks: tasks(
    projectId: $projectId
    filter: {
      status: [TODO]
    }
    first: 10
  ) {
    nodes {
      id
      title
    }
  }

  doneTasks: tasks(
    projectId: $projectId
    filter: {
      status: [DONE]
    }
    first: 10
  ) {
    nodes {
      id
      title
    }
  }
```

```
}
```

4. Response của Alias

Response:

```
{
  "data": {
    "todoTasks": {
      "nodes": [
        {
          "id": "1",
          "title": "Build API"
        }
      ]
    },

    "doneTasks": {
      "nodes": [
        {
          "id": "2",
          "title": "Setup Devise"
        }
      ]
    }
  }
}
```

Client dùng:

```
response.data.data.todoTasks.nodes
```

và:

```
response.data.data.doneTasks.nodes
```

5. Alias nằm ở đâu?

Alias nằm trực tiếp trong GraphQL document:

```
todoTasks: tasks(...)
```

Client gửi document này bằng Axios:

```
const query = `
  query GetTaskGroups($projectId: ID!) {
    todoTasks: tasks(
      projectId: $projectId
      filter: {
        status: [TODO]
      }
      first: 10
    ) {
      nodes {
        id
        title
      }
    }

    doneTasks: tasks(
      projectId: $projectId
      filter: {
        status: [DONE]
      }
      first: 10
    ) {
      nodes {
        id
        title
      }
    }
  }
`;
```

6. Axios request hoàn chỉnh

```
import axios from "axios";

const query = `
  query GetTaskGroups($projectId: ID!) {
    todoTasks: tasks(
      projectId: $projectId
      filter: {
        status: [TODO]
      }
      first: 10
    ) {
      nodes {
        id
        title
        status
        priority
      }
    }

    inProgressTasks: tasks(
      projectId: $projectId
      filter: {
        status: [IN_PROGRESS]
      }
      first: 10
    ) {
      nodes {
        id
        title
        status
        priority
      }
    }

    doneTasks: tasks(
      projectId: $projectId
      filter: {
        status: [DONE]
      }
      first: 10
    ) {
      nodes {
        id
        title
        status
        priority
      }
    }
  }
`;

const response = await axios.post(
  "http://localhost:3000/graphql",
  {
    query,

    variables: {
      projectId: "10"
    },

    operationName: "GetTaskGroups"
  },
  {
    withCredentials: true
  }
);
```

7. Client render thế nào?

```
const todoTasks =
  response.data.data.todoTasks.nodes;

const inProgressTasks =
  response.data.data.inProgressTasks.nodes;
```

```
const doneTasks =
  response.data.data.doneTasks.nodes;
```

React:

```
function TaskBoard({ data }) {
  const todoTasks =
    data.todoTasks.nodes;

  const inProgressTasks =
    data.inProgressTasks.nodes;

  const doneTasks =
    data.doneTasks.nodes;

  return (
    <div>
      <section>
        <h2>TODO</h2>

        {todoTasks.map((task) => (
          <div key={task.id}>
            {task.title}
          </div>
        ))}
      </section>

      <section>
        <h2>IN PROGRESS</h2>

        {inProgressTasks.map((task) => (
          <div key={task.id}>
            {task.title}
          </div>
        ))}
      </section>

      <section>
        <h2>DONE</h2>

        {doneTasks.map((task) => (
          <div key={task.id}>
            {task.title}
          </div>
        ))}
      </section>
    </div>
  );
}
```

8. Alias + Fragment

Bước 35 kết hợp rất tốt với Bước 36.

Fragment:

```
fragment TaskSummaryFields on Task {
  id
  title
  status
  priority
  dueDate
}
```

Query:

```
query GetTaskBoard($projectId: ID!) {
  todoTasks: tasks(
    projectId: $projectId
    filter: {
      status: [TODO]
    }
    first: 10
  ) {
    nodes {
      ...TaskSummaryFields
    }
```

```
  }

  inProgressTasks: tasks(
    projectId: $projectId
    filter: {
      status: [IN_PROGRESS]
    }
    first: 10
  ) {
    nodes {
      ...TaskSummaryFields
    }
  }

  doneTasks: tasks(
    projectId: $projectId
    filter: {
      status: [DONE]
    }
    first: 10
  ) {
    nodes {
      ...TaskSummaryFields
    }
  }
}
```

9. Alias + Variables

Alias không thay Variables.

Ví dụ:

```
query CompareProjects(
  $firstId: ID!
  $secondId: ID!
) {
  firstProject: project(
    id: $firstId
  ) {
    id
    name
  }

  secondProject: project(
    id: $secondId
  ) {
    id
    name
  }
}
```

Variables:

```
{
  "firstId": "10",
  "secondId": "20"
}
```

Ở đây:

```
firstProject
secondProject
→ Alias

$firstId
$secondId
→ Variables
```

10. Alias + Mutation

Alias cũng dùng cho Mutation.

Ví dụ:

```
mutation CreateTwoTasks(
```

```
  $first: CreateTaskInput!
  $second: CreateTaskInput!
) {
  firstTask: createTask(
    input: $first
  ) {
    task {
      id
      title
    }

    errors
  }

  secondTask: createTask(
    input: $second
  ) {
    task {
      id
      title
    }

    errors
  }
}
```

Variables:

```
{
  "first": {
    "projectId": "10",
    "attributes": {
      "title": "Backend"
    }
  },

  "second": {
    "projectId": "10",
    "attributes": {
      "title": "Frontend"
    }
  }
}
```

Response:

```
{
  "data": {
    "firstTask": {
      "task": {
        "id": "1",
        "title": "Backend"
      },
      "errors": []
    },

    "secondTask": {
      "task": {
        "id": "2",
        "title": "Frontend"
      },
      "errors": []
    }
  }
}
```

11. Cẩn thận với nhiều mutation aliases

GraphQL cho phép:

```
firstTask: createTask(...)
secondTask: createTask(...)
```

nhưng đừng coi đây là transaction database.

Nếu:

```
firstTask
→ success

secondTask
→ fail
```

không có nghĩa firstTask tự rollback.

Nếu business cần:

```
tạo 10 Task
→ hoặc t□t c□ thành công
→ hoặc rollback toàn bộ
```

thì nên tạo riêng một mutation/service:

```
createTasksBatch
```

và dùng:

```
ActiveRecord::Base.transaction
```

12. Alias có làm giảm số request không?

Có thể giảm HTTP request count.

Ví dụ thay vì:

```
Request 1
→ TODO tasks

Request 2
→ DONE tasks
```

ta gửi một GraphQL request:

```
Request 1
→ todoTasks
→ doneTasks
```

Nhưng điều này không đồng nghĩa giảm số SQL.

13. Alias có làm giảm SQL không?

Không tự động.

Query:

```
todoTasks: tasks(...)
doneTasks: tasks(...)
```

có thể dẫn đến:

```
todoTasks
→ TasksResolver
→ SQL query 1

doneTasks
→ TasksResolver
→ SQL query 2
```

Alias chỉ giúp:

```
1 HTTP GraphQL request
```

không có nghĩa:

```
1 SQL query
```

14. Alias và Dashboard

Ví dụ bạn có thể làm:

```
query GetTaskDashboard(
  $projectId: ID!
) {
  todoTasks: tasks(
    projectId: $projectId
    filter: {
```

```
      status: [TODO]
    }
    first: 5
  ) {
    nodes {
      id
      title
    }
  }

  overdueTasks: tasks(
    projectId: $projectId
    filter: {
      overdue: true
    }
    first: 5
  ) {
    nodes {
      id
      title
    }
  }
}
```

Điều này hợp lý nếu bạn chỉ cần vài nhóm Task nhỏ.

Nhưng nếu dashboard cần:

```
totalProjects
todoCount
doneCount
overdueCount
10 loại statistics
```

thì một dedicated field:

```
dashboard {
  ...
}
```

thường tốt hơn vì backend có thể aggregate SQL hiệu quả hơn.

15. Alias có đổi schema không?

Không.

Ví dụ schema:

```
field :name,
      String,
      null: false
```

Client:

```
projectTitle: name
```

Schema vẫn là:

```
name
```

Introspection vẫn thấy:

```
name: String!
```

Alias chỉ thuộc operation của client.

16. Alias có dùng để bảo mật không?

Không.

Không bao giờ kiểm tra:

```
if alias_name == "adminTasks"
```

để authorize.

Client tự đặt alias:

```
totallySafe: deleteProject(...)
```

Alias không có giá trị security.

Authorization vẫn phải dùng:

```
current_user
Policy
Policy::Scope
```

17. Alias khác Fragment thế nào?

```
Alias
→ đ□i response key
→ hoặc gọi cùng field nhi□u l□n

Fragment
→ reuse selection set
```

Ví dụ:

```
todoTasks: tasks {
  nodes {
    ...TaskFields
  }
}
```

Ở đây:

```
todoTasks
→ Alias

...TaskFields
→ Fragment
```

18. Alias khác Operation Name thế nào?

```
query GetTaskBoard {
```

GetTaskBoard:

```
Operation Name
```

Còn:

```
todoTasks: tasks
```

todoTasks:

```
Alias
```

Hai thứ hoàn toàn khác nhau.

19. Alias khác Variable thế nào?

```
todoTasks: tasks(
  projectId: $projectId
)
```

Ở đây:

```
todoTasks
→ Alias

$projectId
→ Variable
```

Alias thay đổi tên response.

Variable mang dynamic value.

20. Sơ đồ Bước 36

```
CLIENT
│
├── Operation
│   GetTaskBoard
│
├── Variables
```

```
│   projectId
│
├── Alias
│   todoTasks
│   doneTasks
│
└── Fragment
    TaskFields
        │
        ▼
GraphQL document
        │
        ▼
POST /graphql
        │
        ▼
Rails GraphQL Schema
        │
        ▼
tasks field
        │
        ├── todoTasks
        │    └── TasksResolver
        │
        └── doneTasks
             └── TasksResolver
        │
        ▼
GraphQL Response
        │
        ├── todoTasks
        └── doneTasks
```

Ghi nhớ Bước 36

```
Alias
→ thuộc GraphQL

Được vi□t □ đâu?
→ client GraphQL document

Rails c□n tạo Alias class?
→ không

Dùng đ□ làm gì?
→ đ□i response key
→ gọi cùng field nhi□u l□n với arguments khác nhau

Có dùng trong Query?
→ có

Có dùng trong Mutation?
→ có

Có gi□m SQL?
→ không tự động

Có dùng cho authorization?
→ không
```

Ví dụ thực tế nhất của project này:

```
query GetTaskBoard($projectId: ID!) {
  todoTasks: tasks(
    projectId: $projectId
    filter: {
      status: [TODO]
    }
    first: 10
  ) {
    nodes {
      ...TaskSummaryFields
    }
  }

  doneTasks: tasks(
```

```
    projectId: $projectId
    filter: {
      status: [DONE]
    }
    first: 10
  ) {
    nodes {
      ...TaskSummaryFields
    }
  }
}
```

Trong đó:

```
GetTaskBoard
→ Operation Name

$projectId
→ Variable

todoTasks / doneTasks
→ Aliases

...TaskSummaryFields
→ Fragment
```

Đây chính là cách Bước 33–36 ghép lại thành một GraphQL client request thực tế.